

Cours 02 – Notion 4 : Les méthodes (accesseurs, services et redéfinition)

2.4.1 Définitions

Puisque la technique d'encapsulation est utilisée lors de la création d'une classe. Les propriétés de cette classe ne sont accessibles qu'à l'intérieur des méthodes de cette même classe. L'application dans laquelle cette classe est utile devra pouvoir manipuler ces propriétés privées.

Pour y arriver, quatre groupes de méthodes publiques seront définis :

- **Les méthodes mutatrices** : Elles permettent de modifier la valeur d'une propriété (set).
- **Les méthodes observatrices** : Elles permettent de consulter la valeur d'une propriété (get).
- **Les méthodes de redéfinition** : Elles permettent de modifier le comportement de fonction et opérateur standard sur des objets de types différents.
- **Les méthodes de services** : Elles permettent d'effectuer les calculs et actions plus complexes sur les propriétés d'une classe.

De plus, nous pouvons définir deux groupes de méthode privées:

- **Les méthodes de services** : Elles permettent de factoriser l'écriture des services publics.

Dans tous les cas, l'emploi de méthodes privées sert seulement à améliorer la qualité du code écrit dans les méthodes publiques.

2.4.2 Mise en contexte

Prenons un paquet de carte à jouer. Dans un paquet, nous retrouvons 52 cartes. En voici quelques instances :



```
carte1 :   force = 8
           suite = 4   (trèfle)

carte2 :   force = 9
           suite = 3   (carreau)

carte4 :   force = 12  (dame)
           suite = 2   (pique)

carte5 :   force = 13  (roi)
           suite = 4   (carreau)
```

On remarque que chaque instance (carte1 à carte5) de la classe "Carte" possède deux propriétés. Puisque les propriétés de cette classe sont privées, si l'on désire change la carte1 et la carte4 de notre main, nous ne pouvons pas passer directement par celles-ci.

```
public class Carte
{
    public enum Couleur {coeur, pique, carreau, trefle};
    private int    force;
    private Couleur suite;
}
public class Application
{
    public static void main(String[] args)
    {
        Carte carte1 = new Carte();
        carte1.suite = Carte.Couleur.carreau;
    }
}
```

C:\Users\Francis\Desktop\INF111\PremierProgrammage\src\Application.java:6:15
java: suite has private access in Carte

2.4.3 Les méthodes mutatrices

Tel que mentionné en définition, les mutateurs d'une classe servent à attribuer une nouvelle valeur à une propriété d'une instance de la classe.

Voici leur syntaxe de base:

```
// Syntaxe de déclaration d'un mutateur.  
//  
public void setNomPropriete(nouvelleValeur)  
{  
    this.nomPropriete = nouvelleValeur;  
}
```

Lors d'un appel de méthode, on préfixe le nom de la méthode par l'instance sur laquelle cette méthode s'applique:

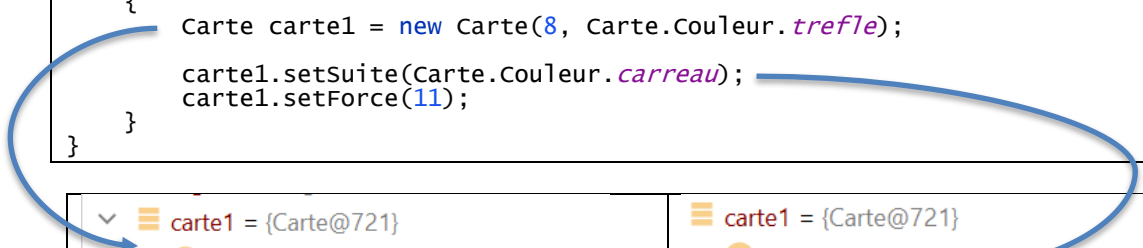
```
// Syntaxe d'appel d'un mutateur.  
//  
nomInstance = Constructeur();  
nomInstance.setNomPropriete(nouvelleValeur);
```

En reprenant l'exemple de mise en contexte défini à la section 2.4.2, voici le code des mutateurs de la classe "Carte".

```
public class Carte  
{  
    public enum Couleur {coeur, pique, carreau, trefle};  
    private int force;  
    private Couleur suite;  
  
    public Carte(int force, Couleur suite)  
    {  
        this.force = force;  
        this.suite = suite;  
    }  
  
    public void setForce(int nouvelleForce)  
    {  
        this.force = nouvelleForce;  
    }  
  
    public void setSuite(Couleur nouvelleCouleur)  
    {  
        this.suite = nouvelleCouleur;  
    }  
}
```

Ainsi, il est maintenant possible de modifier les propriétés d'une carte.

```
public class Application  
{  
    public static void main(String[] args)  
    {  
        Carte carte1 = new Carte(8, Carte.Couleur.trefle);  
        carte1.setSuite(Carte.Couleur.carreau);  
        carte1.setForce(11);  
    }  
}
```



<pre>☰ carte1 = {Carte@721} f force = 0 (0x0) > f suite = {Carte\$Couleur@722} "trefle"</pre>	<pre>☰ carte1 = {Carte@721} f force = 11 (0xB) > f suite = {Carte\$Couleur@829} "carreau"</pre>
--	--

2.4.4 Les méthodes observatrices

Tel que mentionné en définition, les observateurs d'une classe servent à consulter la valeur d'une propriété d'une instance de la classe.

Voici leur syntaxe de base:

```
// Syntaxe de déclaration d'un observateur.  
//  
public typeAttribut getNomPropriete()  
{  
    return this.nomPropriete;  
}
```

Voici la syntaxe utilisée par appeler ce type de méthode:

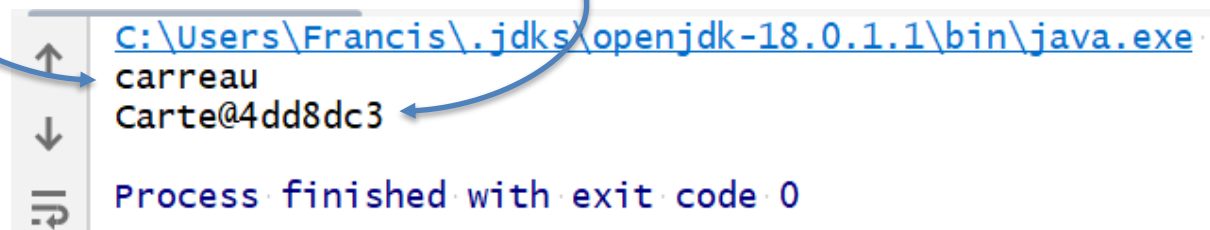
```
//  
// Syntaxe d'appel d'un observateur.  
//  
nomInstance = Constructeur();  
valeur      = nomInstance.getNomPropriete();
```

En reprenant l'exemple de défini à la section 2.4.2, voici le code des observateurs de la classe "Carte".

```
public class Carte  
{  
    ...  
    public int getForce()  
    {  
        return this.force;  
    }  
    public Couleur getSuite()  
    {  
        return this.suite;  
    }  
}
```

Ainsi, il est maintenant possible de visualiser les propriétés d'une carte.

```
public class Application  
{  
    public static void main(String[] args)  
    {  
        Carte carte1 = new Carte(8, Carte.Couleur.trefle);  
        carte1.setSuite(Carte.Couleur.carreau);  
        carte1.setForce(11);  
        System.out.println(carte1.getSuite());  
        System.out.println(carte1);  
    }  
}
```



```
C:\Users\Francis\.jdk\openjdk-18.0.1.1\bin\java.exe  
carreau  
Carte@4dd8dc3  
Process finished with exit code 0
```

On remarque que Java affiche drôlement la carte malgré la présence d'observateur. Il est possible de consulter les données d'une carte, mais pas de l'afficher directement. La redéfinition de méthodes et d'opérateurs servira à contourner ce problème.

2.4.5 La redéfinition de méthodes

Tel que mentionné en définition, la redéfinition de méthode permet de modifier le comportement de méthode standard lorsqu'e l'on manipule des objets de types différents.

Il n'est pas rare, dans un langage de programmation, qu'un nom de méthode ne puisse être utiliser qu'une seule fois. Nous avons vu à la section 2.2.3 que ce n'est pas le cas en Java. Ce langage de programmation est capable de définir et de reconnaître plusieurs fonctions qui porte le même nom, si leur nombre de paramètre diffèrent; c'est ce que l'on a appelé la surcharge de méthode.

La redéfinition de méthode est un principe similaire, mais dans lequel on se sert du type des paramètres pour reconnaître une méthode plutôt que leur nombre.

Cas d'utilisation	Illustration d'une surcharge	Illustration d'une redéfinition
L'on désire obtenir la valeur minimale parmi plusieurs données.	<pre>int min(int v1, int v2); int min(int v1, int v2, int v3); int min(int v1, int v2, int v3, int v4);</pre> C'est trois fonctions font la même action et différent par leurs nombres de paramètres. La logique est la même dans les trois fonctions, elle ne fait que s'attendre à plus de paramètres.	<pre>int meilleur(Etudiant e1, Etudiant e2); int meilleur(Vin v1, Vin v1); int meilleur(Voiture v1, Voiture v2);</pre> C'est trois fonctions font la même action, mais différent par le type de leurs paramètres. La logique diffère assurément, même si les fonctions calculs un concept semblable.

Les deux méthodes qui sont pratiquement toujours redéfinies sont les suivantes :

Cas d'utilisation et explication	Fonction à surcharger
<pre>// L'on désire voir apparaître : "As de pique" Carte carte1 = new Carte(1, Carte.Couleur.pique); System.out.println(carte1);</pre> Lors de l'appel d'un <code>System.out.println</code>, Java appelle automatique la fonction <code>toString</code> pour transformer un objet en texte. C'est notre responsabilité de déterminer comment afficher le contenu d'un objet en redéfinissant la fonction <code>toString</code>.	<pre>String toString() //Code qui transforme this //en String end</pre>
<pre>// L'on désire comparer le contenu de deux cartes Carte carte1 = new Carte(1, Carte.Couleur.pique); Carte carte2 = new Carte(1, Carte.Couleur.pique); if(carte1 == carte2) System.out.println("pareil"); else System.out.println("Different");</pre> Lors de la comparaison, Java compare les références (des no d'identification) des deux objets et non par leur contenu. Ceci n'est pas le comportement voulu, on doit redéfinir la comparaison.	<pre>boolean equals(Carte autre) //Code qui compare this //et autre end</pre>

```
// L'on désire utiliser les "container" de l'API de
// java, ceux-ci utiliser des fonctions mathématiques
// pour représenter les objets : les fonctions de
// hachage.
```

```
Carte carte1 = new Carte(1, Carte.Couleur.pique);
Carte carte2 = new Carte(1, Carte.Couleur.coeur);

HashSet<Carte> mainPoker = new HashSet<Carte>();
mainPoker.add(carte1);
mainPoker.add(carte2);
```

```
boolean estPresent = mainPoker.contains(carte1)
```

Lors de cette recherche, Java ne compare pas les références, ni même les contenus des objets, mais bien une représentation mathématique des objets. Pour ce faire, on doit redéfinir la fonction de hachage.

```
int hashCode()
//Code qui calcule la
// représentation math
// de this.
end
```

Il est important de se rappeler que c'est rarement un programmeur qui appellera les fonctions surcharger. Il s'agit plutôt de Java qui le fait au moment opportun.

```
import java.util.HashSet;

public class Application
{
    public static void main(String[] args)
    {
        Carte carte1 = new Carte(1, Carte.Couleur.pique);
        Carte carte2 = new Carte(1, Carte.Couleur.coeur);

        System.out.println(carte1);           //Appel implicite de toString.
        System.out.println(carte2);           //Appel implicite de toString.

        HashSet<Carte> mainPoker = new HashSet<Carte>();
        mainPoker.add(carte1);
        mainPoker.add(carte2);

        //Appel implicite de equals et possiblement de hashCode
        if (mainPoker.contains(carte1))
            System.out.println("J'ai l'as de pique!");

        if (carte1 == carte2)                  //Appel implicite de equals.
            System.out.println("Nous avons une paire!");
    }
}
```

Voici le code des méthodes surcharger pour la classe "Carte".

```
public String toString()
{
    String forceTxt = this.convertirForce();
    String suiteTxt = String.valueOf(this.suite);

    return forceTxt + " de " + suiteTxt;
}

private String convertirForce()
{
    switch(this.force) {
        case 1 :
            return "As";

        case 11 :
            return "Valet";

        case 12 :
            return "Dame";

        case 13 :
            return "Roi";

        default :
            return Integer.toString(this.force);
    }
}
```

```
public int hashCode()
{
    System.out.println("hc called");
    return Objects.hash(this.force, this.suite);
}

public boolean equals(Object autreCarte)
{
    if(this == null)
        return false;

    if(autreCarte == null)
        return false;

    Carte c = (Carte) autreCarte;
    return (this.force == c.force &&
            this.suite == c.suite);
}
}
```